String Manipulation and Regular Expressions in R (1)

Chapter 4

Guani Wu

Introduction

Most of statistical computing involves working with numeric data. However, many modern applications have considerable amounts of data in the form of text.

There are whole areas of statistics and machine learning devoted to organizing and interpreting text-based data, such as textual data analysis, linguistic analysis, text mining, sentiment analysis, and natural language processing (NLP).

For more information and resources:

- https://cran.r-project.org/web/views/NaturalLanguageProcessing.html
- https://www.tidytextmining.com/

Text-based analyses are beyond the scope of this course.

Introduction

Even in non-text-based analyses, working with data in R often requires processing of characters, such as in row/column names, dates, monetary quantities, longitude/latitude, etc.

Other common scenarios involving characters:

- Removing a given character in the names of your variables
- Changing the level(s) of a categorical variable
- Replacing a given character in a dataset
- Converting labels to upper or lower case
- Extracting a regular pattern of characters from a large text file
- Parsing input from an XML or HTML file

A basic understanding of character (or string) manipulation and regular expressions can be a valuable skill for any statistical analysis.

Characters in R

Symbols in R that represent text or words are called **characters**.

A **string** is a character variable that contains one or more characters, but we often will use "character" and "string" interchangeably.

Values that are stored as characters have base type **`character`** and are typically printed with quotation marks.

```
x <- "Pawnee rules"
x
```

```
## [1] "Pawnee rules"
```

```
typeof(x)
```

```
## [1] "character"
```

Creating Character Strings

Characters can be created using single or double quotation marks. Single quotation marks can be used within double quotation marks and vice versa, but you cannot directly insert single quotes within single quotes or double quotes within double quotes.

```
"This is the 'R' Language"
```

```
## [1] "This is the 'R' Language"
```

```
'This is the "R" Language'
```

```
## [1] "This is the \"R\" Language"
```

```
"This is an "error""
```

```
## Error: unexpected symbol in ""This is an "error"
```

Note: The double quotation inside a string is a special character, which is why it prints with a backslash \".

Empty Characters

The `character()` function creates a character vector of a specified length. The default value in each element of the vector is the **empty character** `""`.

```
character(5)
```

```
## [1] "" "" "" "" ""
```

Note: The empty character `""` is **not the same** as `character(0)`.

The paste() Function

The **paste()** function is one of the most important functions for creating and building strings.

The paste() function inputs one or more R objects, converts them to character, and then concatenates (pastes) them to form one or several character strings.

The basic syntax is:

```
paste(..., sep=" ", collapse = NULL)
```

- The ... argument means the input can be any number of objects.
- The optional **sep** argument specifies the separator between characters after pasting. The default is a single whitespace " ".
- The optional **collapse** argument specifies characters to separate the result.

The paste() Function

```
paste("I ate some", pi, "and it was deloicious.")
```

```
## [1] "I ate some 3.14159265358979 and it was deloicious."
```

```
paste("Bears", "Beets", "Battlestar Galactica", sep = ", ")
```

```
## [1] "Bears, Beets, Battlestar Galactica"
```

```
paste("h", c("a", "e", "o"), sep = "") # No collapsing
```

```
## [1] "ha" "he" "ho"
```

```
paste("h", c("a","e","o"), sep = "", collapse = ", and ")
```

```
## [1] "ha, and he, and ho"
```

Note: Vectors of different lengths will be recycled in the usual way, but partial recycling will not throw a warning.

Print Functions for Characters

There are several functions to print strings:

- `print()` is for generic printing.
- `noquote()` is for printing without quotation marks.
- `cat()` is for concatenation.
- `format()` is for (pretty) printing with special formatting.

The `print()` and `noquote()` Functions

The **`print()`** function (technically the `print.default()` method) has an optional logical `quote` argument that specifies whether to print characters with or without quotation marks.

A similar output can be produced using **`noquote()`**.

```
print(x, quote = FALSE)
```

```
## [1] Pawnee rules
```

```
noquote(x)
```

```
## [1] Pawnee rules
```

Note: While the output appears identical, the commands are not the same. The `noquote()` function outputs a `noquote` class object, which is then inputted into the `print.noquote()` method.

The cat() Function

The **cat()** funtion concatenates multiple character vectors into a single vector, adds a specified separator, and prints the result (without quotations).

```
cat(x, "Eagleton drools", sep = ", ")
```

```
## Pawnee rules, Eagleton drools
```

The printing is slightly different from that of `noquote()`. In particular, the printed output does not have the vector index, and the `cat()` function returns an invisible `NULL` (meaning assigning the printed output to a variable does not work).

The cat() Function

One benefit of `cat()` is that the printed output can be saved to an external file using the **`file`** argument:

```
cat(x, "Eagleton drools", sep = ", ", file = "pawnee.txt")
```

When `file` is specified, an optional logical argument **`append`** specifies whether the result should be appended to or overwrite an existing file.

Side Note: There are a few other optional arguments that are useful for longer text strings. Consult the R documentation for more information.

The `format()` Function

The **`format()`** function formats an R object for "pretty" printing.

Some useful arguments used in `format()`:

- **`width`** specifies the (minimum) width of strings produced.
- **`trim`** specifies whether there should be no padding with spaces (`TRUE`).
- **`justify`** controls how padding takes place for strings. Takes the values `"left"`, `"right"`, `"centre"`, or `"none"`.

For controling the printing of numbers:

- **`digits`** specifies the number of significant digits to use.
- **`nsmall`** specifies the minimum number of digits to the right of the decimal point to include.
- **`scientific`** specifies whether to use scientific notation (`TRUE`) or standard notation (`FALSE`).

The format() Function

```
format(1 / (1 : 5), digits = 2)
```

```
## [1] "1.00" "0.50" "0.33" "0.25" "0.20"
```

```
format(1 / (1 : 5), digits = 2, scientific = TRUE)
```

```
## [1] "1.0e+00" "5.0e-01" "3.3e-01" "2.5e-01"
## [5] "2.0e-01"
```

```
format(c("Pawnee", "rules", "Eagleton", "drools"),
       width = 10, justify = "left")
```

```
## [1] "Pawnee    " "rules     " "Eagleton  "
## [4] "drools    "
```

Functions for Basic String Manipulation

There are many functions in base R for basic string manipulation.

Function	Description
`nchar()`	Returns number of characters
`tolower()`	Converts to lower case
`toupper()`	Converts to upper case
`casefold()`	Wrapper for `tolower()` and `toupper()`
`chartr()`	Translates characters
`substr()`	Extracts substrings of a character vector
`strsplit()`	Splits strings into substrings

Examples of Basic String Manipulation

```
y <- c("Pawnee", "rules", "Eagleton", "drools")
nchar(y)
```

```
## [1] 6 5 8 6
```

```
tolower(y)
```

```
## [1] "pawnee"   "rules"    "eagleton" "drools"
```

```
toupper(y)
```

```
## [1] "PAWNEE"   "RULES"    "EAGLETON" "DROOLS"
```

```
casefold(y, upper = TRUE)
```

```
## [1] "PAWNEE"   "RULES"    "EAGLETON" "DROOLS"
```

Examples of Basic String Manipulation

```
y
```

```
## [1] "Pawnee"   "rules"    "Eagleton" "drools"
```

```
chartr("e", "E", y)
```

```
## [1] "PawnEE"   "rulEs"    "EaglEton" "drools"
```

```
chartr("PE", "pe", y)
```

```
## [1] "pawnee"   "rules"    "eagleton" "drools"
```

```
chartr("aeo", "#?!", y)
```

```
## [1] "P#wn??"   "rul?s"    "E#gl?t!n" "dr!!ls"
```

Examples of Basic String Manipulation

```
x
```

```
## [1] "Pawnee rules"
```

```
substr(x, 2, 9)
```

```
## [1] "awnee ru"
```

```
strsplit(x, split = "l")
```

```
## [[1]]
## [1] "Pawnee ru" "es"
```

```
strsplit(x, split = " ")
```

```
## [[1]]
## [1] "Pawnee" "rules"
```

The stringr Package

The **stringr** package is part of the core tidyverse: it is automatically loaded by typing `library(tidyverse)`.

```
library(tidyverse)
```

The functions in `stringr` are intended to make R's string functions more consistent, simpler, and easier to use. The primary ways it does this are:

- Function and argument names (and positions) are consistent.
- All functions deal with `NA`s and zero length characters appropriately.
- The output data structures from each function matches the input data structures of other functions.

Basic `stringr` Functions

The `stringr` package has functions for basic manipulation and for regular expression operations.

The main `stringr` functions for basic manipulation are shown below.

Function	Description	Similar to
`str_length()`	Number of characters	`nchar()`
`str_c()`	String concatenation	`paste()`
`str_sub()`	Extracts substrings	`substr()`
`str_dup()`	Duplicates characters	None
`str_trim()`	Removes leading and trailing whitespace	`trimws()`
`str_pad()`	Pads a string	None
`str_wrap()`	Wraps a string paragraph	`strwrap()`

More examples and documentation can be found in:

- https://www.gastonsanchez.com/r4strings/stringr-basics.html
- https://cran.r-project.org/web/packages/stringr/vignettes/stringr.html

The str_length() Function

The **str_length()** function works with **factors**, whereas nchar() does not.

```
ordered(month.name)
```

```
##  [1] January   February  March     April
##  [5] May       June      July      August
##  [9] September October   November  December
## 12 Levels: April < August < ... < September
```

```
nchar(ordered(month.name))
```

```
## Error in nchar(ordered(letters)) : 'nchar()'
requires a character vector‘
```

```
str_length(ordered(month.name))
```

```
##  [1] 7 8 5 5 3 4 4 6 9 7 8 8
```

The `str_c()` Function

The **`str_c()`** function automatically removes `NULL` and `character(0)` arguments.

```
paste(x, NULL, character(0), c("Eagleton drools"), sep=", ")
```

```
## [1] "Pawnee rules, , , Eagleton drools"
```

```
str_c(x, NULL, character(0), c("Eagleton drools"), sep=", ")
```

```
## character(0)
```

Note: The default separator for `paste()` is whitespace (`sep=" "`) and for `str_c()` is the empty string (`sep=""`).

The str_c() Function (Cont.)

```
name <- "Hadley"
time_of_day <- "morning"
birthday <- TRUE

str_c(
  "Good ", time_of_day, " ", name,
  if (birthday) " and HAPPY BIRTHIDAY",
  "."
)
```

```
## [1] "Good morning Hadley and HAPPY BIRTHIDAY."
```

Regular Expressions

One main application of string manipulation is pattern matching. Finding patterns in text are useful for data validation, data scraping, text parsing, filtering search results, etc.

A **regular expression** (or **regex**) is a set of symbols that describes a text pattern. More formally, a regular expression is a pattern that describes a set of strings.

Regular expressions are a formal language in the sense that the symbols have a defined set of rules to specify the desired patterns. The best way to learn the syntax and become fluent with regular expressions is to practice.

Applications of Regular Expressions

Some common applications of regular expressions:

- Test if a phone number has the correct number of digits
- Test if a date follows a specifc format (e.g. mm/dd/yy)
- Test if an email address is in a valid format
- Test if a password has numbers and special characters
- Search a document for gray spelled either as “gray” or “grey”
- Search a document and replace all occurrences of “Will”, “Bill”, or “W.” with “William”
- Count the number of times in a document that the word “analysis” is immediately preceded by the words “data”, “computer”, or “statistical”
- Convert a comma-delimited file into a tab-delimited file
- Find duplicate words in a text

`stringr` Functions for Regular Expressions

R has native regex handling capabilities (e.g. **`grep()`**), but `stringr` has made their usage easier and more consistent.

Function	Description
`str_view(string, pattern)`	shows the first match
`str_detect(string, pattern)`	Detect the presence or absence of a pattern in a string and returns `TRUE` if it is found
`str_locate(string, pattern)`	Locate the first position of a pattern and returns a matrix with start and end.
`str_extract(string, pattern)`	Extracts text corresponding to the first match.
`str_match(string, pattern)`	Extracts capture groups formed by `()` from the first match.
`str_split(string, pattern)`	Splits string into pieces and returns a list of character vectors.

Many of these functions have variants with an `_all` suffix which will match more than one occurrence of the pattern in a given string.

Literal Characters

The most basic type of regular expressions are **literal characters**, which are characters that match themselves.

A literal character match is one in which a given character such as the letter `"R"` matches the letter R. This type of match is the most basic type of regular expression operation: just matching plain text.

All the letters and digits in the English alphabet (i.e., alphanumeric characters) are considered literal characters because, as regular expressions, they match themselves.

Matching Literal Characters

Literal character matching is case sensitive.

```
love_stats <- "I love stats, so I am a Stats major."
str_view(love_stats, "stat")
```

```
## [1] | I love <stat>s, so I am a Stats major.
```

Matching Literal Characters

The `str_locate()` function only returns the first occurrence of a match. To find all matches, use `str_locate_all()`.

```
love_stats <- "I love statistics, so I am a stats major."
str_locate(love_stats, "stat")
```

```
##      start end
## [1,]     8  11
```

```
str_locate_all(love_stats, "stat")
```

```
## [[1]]
##      start end
## [1,]     8  11
## [2,]    30  33
```

Metacharacters

Not all characters match themselves. Any character that is not a literal character is a **metacharacter**.

The power of regular expressions comes from the ability to use a number of special metacharacters that modify how the pattern matching is performed.

The list of metacharacters used in regular expressions is given below:

```
. ^ $ * + ? { } [ ] \ | ( )
```

The Wild Metacharacter

The **dot** (or **period**) **.** is called the **wild** metacharacter (sometimes the **wildcard**). This metacharacter is used to match ANY (single) character except a new line.

```
nxt <- c("not", "nit", "network", "nut", "n t")
str_view(nxt, "n.t")
```

```
## [1] | <not>
## [2] | <nit>
## [3] | <net>work
## [4] | <nut>
## [5] | <n t>
```

The Wild Metacharacter

Question: Consider the following vector.

```
fives <- c("5.00", "5100", "5-00", "5 00")
```

What will `str_view(fives,"5.00")` return?

The Wild Metacharacter

Question: Consider the following vector.

```
fives <- c("5.00", "5100", "5-00", "5 00")
```

What will `str_view(fives,"5.00")` return?

```
str_view(fives, "5.00")
```

```
## [1] | <5.00>
## [2] | <5100>
## [3] | <5-00>
## [4] | <5 00>
```

Escaping Metacharacters

Because of their special properties, metacharacters cannot be matched directly as literal characters.

To do a literal match, we need to **escape** the metacharacter by adding a backslash \ in front of the metacharacter.

In R, however, since the backlash \ itself is a metacharacter for normal strings, we need a **double backslash** \\ to escape metacharacters in regular expressions.

```
str_view("abc[def", "[")
```

```
## Error in stri_locate_first_regex(string, pattern,
opts_regex = opts(pattern)) :    Missing closing bracket
on a bracket expression. (U_REGEX_MISSING_CLOSE_BRACKET)
```

```
str_view("abc[def", "\\[")
```

```
## [1] | abc<[>def
```

Escaping The Wild Metacharacter

Consider the following vector.

```
fives <- c("5.00", "5100", "5-00", "5 00")
```

To match the literal dot `.` and match only `5.00`, we need to escape the wild metacharacter:

```
str_view(fives, "5\\.00")
```

```
## [1] | <5.00>
```

Character Sets

Square brackets `[ ]` indicate a **character set**, which will match any one of the characters that are inside the set.

A character set will match only one character. The order of the characters inside the set does not matter. For example, the character set defined by `[aeiou]` will match any one lower case vowel.

```
pnx <- c("pan", "pen", "pin", "p0n", "p.n",
         "paun", "pwn3d")
str_view(pnx, "p[aeiou]n")
```

```
## [1] | <pan>
## [2] | <pen>
## [3] | <pin>
```

Character Ranges

To match all capital letters (in English), we can define the character set `[ABCDEFGHIJKLMNOPQRSTUVWXYZ]`.

However, this expression is long and inconvenient. Fortunately, the **dash** `-` metacharacter is a shortcut to indicate a **range** of characters.

Some examples:

Pattern	Character Range
`[a-q]`	All lower case letters from a to q
`[A-Q]`	All upper case letters from A to Q
`[a-zA-Z]`	All 52 ASCII letters
`[0-7]`	All digits from 0 to 7

Character Ranges

Character ranges are useful to match various occurrences of a certain type of character.

Question: Consider the following vector.

```
triplets <- c("123", "abc", "ABC", ":-)")
```

What pattern can be defined to match 3 consecutive digits?

Character Ranges

Character ranges are useful to match various occurrences of a certain type of character.

Question: Consider the following vector.

```
triplets <- c("123", "abc", "ABC", ":-)")
```

What pattern can be defined to match 3 consecutive digits?

```
str_view(triplets, "[0-9][0-9][0-9]")
```

```
## [1] | <123>
```

Similarly, the pattern `[a-z][a-z][a-z]` matches three consecutive lower case letters.

Negative Character Sets

A common situation when working with regular expressions consists of matching characters that are NOT part of a certain set.

This type of matching can be done using a **negative character set**: by matching any one character that is not in the set.

The **caret** `^` metacharacter is used to create negative character sets.

If a caret `^` is placed in the first position inside a character set, it means **negation** (similar to the negative sign in numeric indices or the exclamation point in logical expressions).

For example, the pattern `[^aeiou]` means "not any one of lower case vowels."

Note: The caret `^` is a metacharacter that has more than one meaning depending on where it appears in a pattern.

Negative Character Sets

For example, the pattern `[^A-Z]` will match any character that is NOT an upper case letter.

```
basic <- c("1", "a", "A", "&", "-", "^")
str_view(basic, "[^A-Z]")
```

```
## [1] | <1>
## [2] | <a>
## [4] | <&>
## [5] | <->
## [6] | <^>
```

Negative Character Sets

Caution: It is important that the caret ^ is the first character inside the character set, otherwise the set is not a negative one.

For example, the pattern `[A-Z^]` mean any one upper case letter or the caret character.

```
str_view(basic, "[A-Z^]")
```

```
## [3] | <A>
## [6] | <^>
```

Question: What pattern means "anything except the caret?"

Negative Character Sets

The pattern `[^^]` can be used to mean anything except the caret.

But wait: I thought metacharacters needed to be escaped to be interpreted as literal characters?

```
str_view(basic, "[^^]")
```

```
## [1] | <1>
## [2] | <a>
## [3] | <A>
## [4] | <&>
## [5] | <->
```

Negative Character Sets

```
str_view(basic, "[^\\^]") # same thing
```

```
## [1] | <1>
## [2] | <a>
## [3] | <A>
## [4] | <&>
## [5] | <->
```

Metacharacters Inside Character Sets

Most metacharacters inside a character set are already escaped. This implies that you do not need to escape them using double backslashes.

Not all metacharacters become literal characters when they appear inside a character set. The exceptions are the opening bracket `[`, the closing bracket `]`, the dash `-`, the caret `^` (if at the front or by itself), and the backslash `\`.

The dot `.` inside the character set now represents the literal dot character rather than the wildcard character.

```
str_view(pnx, "p[ae.iou]n")
```

```
## [1] | <pan>
## [2] | <pen>
## [3] | <pin>
## [5] | <p.n>
```

Character Classes

Closely related to character sets and character ranges are **character classes**, which are used to match a certain class of characters.

The most common character classes in most regex engines are:

Pattern	Matches	Same as
`\\d`	Any digit	`[0-9]`
`\\D`	Any non-digit	`[^0-9]`
`\\w`	Any word character	`[a-zA-Z0-9_]`
`\\W`	Any non-word character	`[^a-zA-Z0-9_]`
`\\s`	Any whitespace character	`[\f\n\r\t\v ]`
`\\S`	Any non-whitespace character	`[^\f\n\r\t\v ]`

The whitespace characters are summarized on the following slide.

Character classes can be thought of as another type of metacharacter or as shortcuts for special character sets.

Whitespace

There are several types of whitespace characters, shown in the following table:

Character	Description
`\f`	Form feed (page break)
`\n`	Line feed (new line)
`\r`	Carriage return
`\t`	Tab
`\v`	Vertical tab

For situations with non-printing whitespace characters, it can be difficult to determine which exact character it is, so the whitespace class `\\s` is a useful way to match with all of them.

Character Classes

For example:

```
pnx <- c("pan", "pen", "pin", "p0n", "p.n",
         "paun", "pwn3d")
str_detect(pnx, "p\\d")
```

```
## [1] FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
```

```
str_detect(pnx, "p\\D")
```

```
## [1]  TRUE  TRUE  TRUE FALSE  TRUE  TRUE  TRUE
```

```
str_detect(pnx, "p\\W")
```

```
## [1] FALSE FALSE FALSE FALSE  TRUE FALSE FALSE
```

POSIX Character Classes

There is another type of character classes known as **POSIX character classes** that is supported by the regex engine in R.

The main POSIX classes are:

Class	Description	Same as
`[:alnum:]`	Any letter or digit	`[a-zA-Z0-9]`
`[:alpha:]`	Any letter	`[a-zA-Z]`
`[:digit:]`	Any digit	`[0-9]`
`[:lower:]`	Any lower case letter	`[a-z]`
`[:upper:]`	Any upper case letter	`[A-Z]`
`[:space:]`	Any whitespace, inluding space	`[\f\n\r\t\v ]`
`[:punct:]`	Any punctuation symbol	
`[:print:]`	Any printable character	
`[:graph:]`	Any printable character excluding space	
`[:xdigit:]`	Any hexadecimal digit	`[a-fA-F0-9]`
`[:cntrl:]`	ASCII control characters	

POSIX Character Classes

To use POSIX classes in R, the class needs to be wrapped inside a regex character class, i.e., the class needs to be inside a second set of square brackets.

For example:

```
pnx <- c("pan", "pen", "pin", "p0n", "p.n",
         "paun", "pwn3d")
str_view(pnx, "[[:alpha:]]")
```

```
## [1] | <p><a><n>
## [2] | <p><e><n>
## [3] | <p><i><n>
## [4] | <p>0<n>
## [5] | <p>.<n>
## [6] | <p><a><u><n>
## [7] | <p><w><n>3<d>
```

Anchors

An **anchor** is a pattern that does not match a character but rather a position before, after, or between characters. Anchors are used to "anchor" a match at a certain position.

Pattern	Meaning
`^` or `\\A`	Start of string
`$` or `\\Z`	End of string
`\\b`	Word boundary (i.e., the edge of a word)
`\\B`	Not a word boundary

Anchor Examples

```
text <- "the quick brown fox jumps over the lazy dog dog"
str_view(text, "^the")
```

```
## [1] | <the> quick brown fox jumps over the lazy dog dog
```

Anchor Examples

```
text <- "the quick brown fox jumps over the lazy dog dog"
str_view(text, "\\Athe")
```

```
## [1] | <the> quick brown fox jumps over the lazy dog dog
```

Anchor Examples

```
text <- "the quick brown fox jumps over the lazy dog dog"
str_replace_all(text, "the","-")
```

```
## [1] "- quick brown fox jumps over - lazy dog dog"
```

```
str_replace_all(text, "\\bthe","-")
```

```
## [1] "- quick brown fox jumps over - lazy dog dog"
```

```
str_replace_all(text, "\\Athe","-") # same thing
```

```
## [1] "- quick brown fox jumps over the lazy dog dog"
```

Anchor Examples

```
text <- "the quick brown fox jumps over the lazy dog dog"
str_replace_all(text, "dog","-")
```

```
## [1] "the quick brown fox jumps over the lazy - -"
```

Anchor Examples

```
text <- "words jump jumping umpire pump umpteenth lumps"
str_replace_all(text, "\\bump","-")
```

```
## [1] "words jump jumping -ire pump -teenth lumps"
```

```
str_replace_all(text, "\\Bump","-")
```

```
## [1] "words j- j-ing umpire p- umpteenth l-s"
```

```
str_replace_all(text, "ump\\b","-")
```

```
## [1] "words j- jumping umpire p- umpteenth lumps"
```

```
str_replace_all(text, "ump\\B","-")
```

```
## [1] "words jump j-ing -ire pump -teenth l-s"
```